

Quiz — 1.4.1 Data Types — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Answer key

Section A (8 marks — 1 each) 1. **B** — 1 hex digit = 1 nibble = 4 bits. 2. **C** — $32+8+4+1 = 45$. 3. **B** — MSB in two's complement is the negative place value -128 . 4. **B** — logical left shift n places = $\times 2^n$; left 2 = $\times 4$. 5. **C** — mantissa holds significant figures $\rightarrow$ precision (exponent sets range). 6. **D** — positive normalised mantissa starts $0.1\dots$. 7. **C** — 'A'=65, so 'D' = $65+3 = 68$. 8. **B** — first 128 Unicode code points match ASCII, so ASCII is a subset.

Section B

9. [2] 217 $\rightarrow$ binary. $217 = 128+64+16+8+1$. 128(1) 64(1) 32(0) 16(1) 8(1) 4(0) 2(0) 1(1) $\rightarrow$ check: $128+64+16+8+1 = 217$. $\checkmark$ Answer: **11011001**. Mark: correct method/subtraction (1); correct 8-bit answer (1).

10. [2] 10110110 $\rightarrow$ hex. Split into nibbles from the right: 1011 0110. 1011 = $8+2+1 = 11 = \text{B}$. 0110 = $4+2 = 6 = \text{6}$. Answer: **B6**. Mark: correct nibble split/values (1); correct hex B6 (1).

11. [2] B7 $\rightarrow$ denary. B = 11, 7 = 7. $(11 \times 16) + 7 = 176 + 7 = \text{183}$. Mark: $11 \times 16 = 176$ (1); $+7 = 183$ (1).

12. [4] $45 - 28 = 45 + (-28)$. $+45 = 00101101$ ($32+8+4+1 = 45$). $\checkmark$ $+28 = 00011100$ ($16+8+4 = 28$). $\checkmark$ -28 : invert 00011100 $\rightarrow$ 11100011, then $+1 \rightarrow$ 11100100. Add:

```
00101101    (45)
+ 11100100   (-28)
-----
1 00010001
```

The carry out of the MSB is **discarded** (1 dropped), leaving 00010001. $00010001 = 16 + 1 = \text{17}$. $\checkmark$ ($45 - 28 = 17$) Mark: $+45$ and $+28$ in binary (1); correct two's complement of 28 = 11100100 (1); correct addition (1); discard final carry $\rightarrow$ answer 17 (1).

13. [4] Mantissa 0.0110100, exponent 0011 (= +3). Normalisation rule: a positive number must have a mantissa beginning 0.1... (first significant bit immediately after the point). The current mantissa is 0.0110100 — it has one wasted leading 0 after the point. Shift the point **right by 1 place** to get 0.1101000. Moving the point right $\Rightarrow$ the **exponent decreases** by 1: 0011 (+3) $- 1 =$ 0010 (+2). Normalised mantissa = **0.1101000**, exponent = **0010** (+2). (Value check: original = $0.011010 \times 2^3 = 0.40625 \times 8 = 3.25$; new = $0.110100 \times 2^2 = 0.8125 \times 4 = 3.25$. $\checkmark$ value unchanged.) Mark: identifies mantissa must start 0.1 (1); shifts point right one place $\rightarrow$ 0.1101000 (1); exponent decreases to 0010/+2 (1); states rule/value unchanged (1).

14. [3] 00010110 logical left shift 2 places. Bits move left, 0s fill from the right, top bits lost: 00010110 $\rightarrow$ (1 left) 00101100 $\rightarrow$ (2 left) 01011000. Answer: **01011000**. Effect: multiplied by $2^2 = \times 4$. Check: $00010110 = 22$; $01011000 = 64+16+8 = 88 = 22 \times 4$. $\checkmark$ Mark: correct shifted bits 01011000 (1); 0s fill from right / bits lost stated (1); effect = $\times 4$ ($\times 2^2$) (1).

Section C

15. [5] (a) **[2]** With a fixed total number of bits, giving more bits to the **mantissa** increases **precision** (more significant figures / smaller rounding error) but leaves fewer bits for the exponent, reducing **range** (1). Giving more bits to the **exponent** increases the **range** of values representable but reduces **precision** (1). It is a trade-off — you cannot increase both at once.

(b) **[3]** Storing the number "in hexadecimal" does **not** reduce memory (1): hex is only a human-readable shorthand for the underlying binary — the value is still stored as the same number of bits regardless of whether it is written in hex or binary (1). A genuine reason to use hex: it is a **compact, lossless representation of binary** that is easier for humans to read/write with fewer errors (e.g. memory addresses, colour codes, machine code) — 1 hex digit maps exactly to 4 bits (1).

Total: 8 + 17 + 5 = 30 marks.